

SaaS Platform Boilerplate

Development Brief

Main domain + admin subdomain + customer app subdomain

Written as the base specification I want followed across my SaaS projects

Instruction to developer

All of my SaaS products will follow this same overall structure. Once the final design and architecture are agreed, I want this boilerplate foundation reused across future projects. Product-specific app functionality will change by project, but the account, billing, team, admin, content, operational, and system layers need to be built in a reusable way from the start.

In this document I use test.com, admin.test.com, and app.test.com as placeholders for the live domains.

1. Platform structure I want used on all projects

Every project needs to be split into three connected surfaces. The public website sells and explains the product, the admin area runs the business internally, and the app is where paying users use the service.

Surface	Example domain	Primary users	Purpose
Main website	test.com	Visitors, leads, buyers	Explain the product, build trust, rank in search, capture leads, and convert traffic into signup or purchase.
Admin panel	admin.test.com	Me and my staff	Run the full platform: customers, orders, subscriptions, products, content, emails, settings, affiliates, support, analytics, logs, and system controls.
Customer app	app.test.com	Customers and their team members	Let users register, log in, buy plans, manage accounts, manage teams, manage billing, and use the product itself.

2. Core build rules

Area	Requirement
Reusability	I want this built as a reusable SaaS boilerplate, not as a one-off build. Standard modules must be portable across future products.
Architecture	Use one connected platform with clear separation between the website, admin, and app. Shared services should not be duplicated.
Design consistency	The visual system, spacing, components, and interaction patterns need to feel consistent across all three surfaces while still fitting their purpose.
Modularity	Product-specific app features must sit inside a modular structure so each SaaS product can add custom modules without rewriting the whole base.

Detail level	Admin pages need to be detailed and operational. Where useful, pages should include summary stats at the top and full table/list/detail management below.
Scalability	The structure must handle more users, more products, more plans, more staff, more content, and more system events without needing a rebuild.
Safety	Sensitive actions must be permission-gated, logged, and protected with confirmations where needed.
Documentation	Final delivery must include code, deployment instructions, environment variables, route map, database migrations, and setup notes.

3. Main website scope — test.com

The main website is the public-facing sales and information layer. I want it set up so it can explain the product clearly, convert traffic well, support SEO, and be easy for us to manage without code changes for normal content updates.

Page / module	What I need built	Admin control required	Notes
Homepage	Clear value proposition, CTA sections, feature blocks, testimonials/logos if used, pricing CTA, FAQ, and final conversion section.	Editable key content blocks, hero copy, CTA labels, media, SEO fields.	Built for conversion first, not just design.
Pricing	Show plans, monthly/yearly options if used, free trial if enabled, included credits/usage if relevant, and direct route into signup or checkout.	Manage plans shown, plan order, labels, comparison rows, FAQ, SEO.	Must stay synced with live billing plans.
Feature / solution pages	Dedicated pages for features, use cases, industries, or workflows.	Editable page content, SEO, slugs, visibility, publish state.	Needs reusable content blocks or CMS sections.
About / company pages	About, team, story, mission, contact, or similar trust pages.	Editable content and SEO.	Only include pages actually used on each project, but structure

			must support them.
Blog / resources	Blog, news, knowledge, articles, or resource centre.	Create, edit, schedule, publish, categorise, set SEO, add featured images.	Reusable CMS structure across projects.
Legal pages	Privacy policy, terms, cookie policy, refund policy, DPA or similar.	Editable content and version control or publish history.	Legal page publishing must be simple.
Contact / lead forms	Contact, demo request, consultation request, waitlist, newsletter capture, and any other lead forms.	Manage forms, submissions, notification routing, labels, and status.	All submissions must enter admin.
Auth entry points	Login, register, start trial, join workspace, forgot password, reset password, verify email.	Control which auth options are enabled and where CTAs point.	Public site must route cleanly into app flows.
Localization / translations	Language switcher where required, translated public content, locale-aware slugs if used.	Manage active languages and content per language.	Built so more languages can be added later.
SEO / analytics	Meta title, description, keywords if used, OG data, schema, tracking scripts, pixel/code injection areas.	Edit SEO fields and script settings from admin.	Need clean separation between content and tracking settings.
Changelog / product updates	Public changelog or updates page if used.	Create and publish update posts or release notes.	Useful across most SaaS projects.
Status / trust elements	Optional trust badges, uptime link, security page, integrations page, or partner page.	Editable from admin.	Structure should support these without a custom rebuild.

4. Admin panel scope — [admin.test.com](#)

This needs to be a full internal operations system for me and my workers. It cannot be a shallow back office. Most modules need list views, filters, search, saved views where useful, exports, detail pages, edit/create actions, and clear activity history. Where useful, each page should also have summary cards or mini dashboards at the top.

Admin module	What needs to exist	Key actions	Important rules
Admin authentication	Admin login, password reset, optional SSO/google login, MFA support, session controls.	Log in, log out, reset password, manage sessions, enforce MFA.	Admin access must be separated from normal customer login.
Dashboard	Overview of revenue, subscriptions, signups, churn, failed payments, active users, support load, campaigns, jobs, and recent system activity.	View widgets, charts, alerts, quick links, recent actions.	Widgets can differ by permission.
Admin users	Manage internal users, roles, permissions, status, invitations, and last activity.	Create, invite, edit, disable, reset password, assign roles, view logs.	Permissions must be action-level, not only page-level.
Roles and permissions	Granular RBAC for admin modules and actions.	Create role, clone role, assign permissions, audit changes.	Need permissions for view/create/edit/delete/export/approve/configure .
Customers	Full customer record with account status, plan, workspace data, usage, notes, support history, orders, billing history, and security events.	Search, filter, view, edit safe fields, suspend, restore, impersonate with logging, tag, add notes.	High-risk actions must log actor and timestamp.
Orders and checkouts	View all orders and checkout records across one-off and recurring purchases.	Search, filter, refund, resend invoice, inspect checkout metadata.	Keep clear link to customer and payment records.
Payments and subscriptions	Manage subscriptions, invoices, payment attempts, renewals, cancellations, plan changes, credits, and failed payments.	View records, retry, cancel, comp, issue credit, apply adjustment, pause, resume.	Must support trial, upgrade, downgrade, cancellation, and grace states.
Products, plans, and credits	Create and manage subscription plans, one-time products, bundles, add-ons, credit packs, entitlements, and pricing display settings.	Create, edit, archive, duplicate, reorder, change status.	Product catalogue must support future SaaS products too.

Coupons and promotions	Discount codes, trial extensions, limited offers, referral promos, and campaign-linked offers.	Create, edit, expire, limit usage, apply rules.	Need clear eligibility rules.
Email templates	All transactional email templates used by the system.	Create, edit, preview, test send, enable/disable, localise.	Track which events trigger each template.
Email sending settings	AWS SES and any other sending settings, domains, sender details, reply-to, tracking options.	Add/update config, verify status, choose defaults.	Environment secrets must still be handled safely.
Promotional email campaigns	Newsletter and campaign builder for sending promotional emails to selected audiences.	Create audience segments, schedule, send, view metrics, suppress users.	Respect unsubscribe and consent rules.
Notifications management	Create in-app notices, banners, announcements, and targeted messages.	Create, schedule, target, expire, prioritise.	Need targeting by plan, segment, locale, or workspace.
CMS / site content	Manage the public website content, reusable blocks, menus, footer, legal pages, blog/resources, and other editable content areas.	Create, edit, publish, unpublish, schedule, preview.	This is how normal website content should be maintained.
Translation management	Manage languages, translated strings, translated content, and language visibility.	Add locale, edit translations, enable/disable locale.	Avoid hard-coding text where translation is expected.
Site settings	Site name, logo, favicon, brand assets, meta defaults, contact details, support links, script injection areas, and other platform-wide settings.	Edit and save settings with audit logging.	Separate safe settings from highly sensitive system settings.
Auth settings	Google auth and other auth-provider settings, registration rules, invite rules, email verification	Enable/disable providers, set redirect rules, set registration policy.	Do not expose secrets in plain text.

	rules.		
Payment settings	Stripe, PayPal, and any later gateway settings.	Add config, switch defaults, set gateway behaviour, inspect webhooks.	Support one or multiple gateways.
Affiliate system	Affiliate users, codes/links, referred signups, commissions, statuses, and payouts.	Create, approve, adjust commission, mark payout, export records.	Need fraud-aware notes and audit history.
Tracking and analytics	Track what customers do on the main site and app.	View funnels, events, conversion data, page activity, campaign attribution.	Needs event definitions and clean reporting structure.
API keys and integrations	Store and manage platform API keys, external integration credentials, webhook endpoints, and service configuration.	Create masked records, rotate, disable, test, scope access.	Secrets need safe handling and visibility controls.
Support / helpdesk	Customer issues, tickets, feedback, contact submissions, internal notes, status, assignee, and attachments if used.	Create, assign, reply, tag, close, reopen, export.	Can be native or integrated, but must exist operationally.
App page management	Admin-side control for app text blocks, onboarding steps, notices, help content, and other editable non-code app content.	Edit, publish, target, order.	This covers the requirement for editable app pages.
Audit logs	Complete logs of sensitive actions across admin and app.	Search, filter, inspect actor, target, before/after snapshot where appropriate.	Must exist across all major modules.
Jobs / queues / webhooks	Operational screen for background jobs, scheduled jobs, failed jobs, outgoing webhooks, incoming webhooks, retries, and logs.	Inspect, retry, replay, cancel where safe.	Needed for stable SaaS operations.
Feature flags	Enable or disable features by	Create, toggle, target, document.	Useful for staged rollouts.

	environment, project, plan, or account.		
File and media library	Manage uploaded images, documents, and reusable media assets.	Upload, tag, replace, delete, inspect usage.	Needed for CMS and email content.
System health	Health status, cron/task status, queue depth, storage usage, error summaries, and service alerts.	View status and alerts.	Useful for admin dashboard or separate ops screen.

5. Customer app scope — app.test.com

The app needs to be the reusable customer shell across all projects. Standard account, billing, team, and settings features must be present in every build. Product-specific functionality then sits inside this shell depending on the SaaS product.

App module	What I need built	User actions	Rules
Authentication	Register, log in, Google login, forgot password, reset password, verify email, invite acceptance, and secure session flows.	Sign up, sign in, reset password, verify email, accept invite.	Support both direct signup and invite-based entry.
Onboarding	First-run onboarding flow to guide new users into the right setup path.	Complete onboarding steps, skip where allowed, resume later.	Progress needs to be trackable.
Dashboard	Customer dashboard with account summary, usage summary, plan status, key recent activity, and shortcuts into the product.	View widgets and recent items.	Dashboard content can vary by product, but shell must exist.
Pricing / buy flow	Plan selection, checkout start, trial start, one-time purchases, and buy credits flow.	Choose plan, start trial, buy plan, buy add-on, buy credits.	Needs clean path from public site or inside app.
Profile management	Manage name, email, avatar, company/workspace details, password, and personal preferences.	Edit profile, change password, manage preferences.	Email change flow must be safe.

Subscription management	View current plan, next renewal, billing cycle, trial status, included usage/credits, and available upgrade paths.	Upgrade, downgrade, cancel, resume, reactivate, switch cycle.	Plan changes need clear proration and effective-date logic.
Billing management	Payment method management, invoices, receipts, billing history, tax/business info if used.	Add/update card, download invoice, view billing history.	Need support for failed payment and recovery flow.
Credits / usage	Show available credits, consumed credits, quotas, limits, usage history, and top-up options.	View usage, buy credits, review activity.	This is required where plans include credits or usage caps.
Team management	Invite team members, manage role/access, remove members, and view team activity.	Invite, resend invite, change role, remove, transfer ownership where allowed.	Roles must be controlled by workspace-level permissions.
Workspace settings	Workspace name, branding/settings if relevant, integrations, defaults, API settings, and product-specific workspace controls.	Edit workspace settings.	Module available only where needed per product.
Notifications centre	In-app notifications, announcements, account warnings, billing alerts, and product messages.	View, dismiss, click through, mark read.	Admin-created notices need to appear here.
Help and support	Help centre, docs links, contact support, ticket history, feedback, and bug reports.	Open ticket, view responses, submit feedback.	Must be available inside the app.
Language / locale	Change language and locale preferences where localization is enabled.	Switch language.	Need persistence by account or user.
Security	Session management, MFA if enabled, device/session visibility, and security event history.	Manage sessions, enable MFA, review security events.	Security controls must be easy to find.
API access	Customer API	Create token, rotate,	Only for products/plans

	tokens/keys where the product includes API access.	revoke, view usage.	where API access is included.
Affiliate area	Affiliate account view for customers who act as affiliates.	View referrals, links, commissions, payout status.	If enabled on the project, it must connect to admin affiliate module.
Product modules	The actual SaaS-specific tools, workflows, data screens, generators, editors, dashboards, or automations for that product.	Use the product.	These modules vary by project and plug into this shell.
Changelog / updates	See what changed inside the product.	View release notes or updates.	Can link to public or in-app changelog.
Account lifecycle	Delete account request, close workspace, leave workspace, export account data where required, and reactivate within allowed rules.	Submit lifecycle actions.	These flows need admin visibility and logging.

6. Shared platform services that need to exist across all projects

Shared service	What it needs to cover	Why I need it built into the base
Authentication service	Login, registration, password flows, social login, email verification, invites, admin auth separation, session handling, MFA hooks.	Every project will need this and I do not want it rebuilt each time.
Billing service	Plans, subscriptions, one-time payments, credits, invoices, coupons, renewals, proration, failed payment handling, cancellations.	Billing behaviour must stay consistent across future products.
Entitlements and usage service	Feature access by plan, seat count rules, usage tracking, quota checks, credit balance logic, top-up logic.	This is needed for SaaS plans, add-ons, and credits.
Notification service	Transactional email triggers, in-	Communication logic should be

	app notifications, banners, announcements, campaign send hooks, preference rules.	centralised.
CMS/content service	Editable pages, SEO fields, menus, footer, legal content, changelog, blog/resources, reusable content blocks.	Needed so non-code content can be managed in admin.
Localization service	Languages, string dictionaries, translated content, locale settings, language visibility.	Required for multilingual projects.
Media/file service	Image/document upload, storage references, usage mapping, and safe deletion rules.	Needed across CMS, support, emails, and product modules.
Analytics/event service	Track key events, page views, conversions, plan changes, product activity, attribution, and support operational reporting.	I want consistent event naming and reporting.
Audit/logging service	Actor logging, target logging, before/after where relevant, and searchable logs.	Needed for support, safety, and accountability.
Queue/job/webhook service	Background tasks, retries, scheduled jobs, incoming/outgoing webhook processing, logs, and failure handling.	Modern SaaS products need this built in.
Feature flag service	Roll out features by environment, product, plan, or account.	Needed for staged launches and controlled releases.
Settings/configuration service	Store editable business settings, site settings, and operational settings with permission controls.	Avoid hard-coded changes for normal operations.

7. Roles and permissions model

Permissions need to be more detailed than simple page access. I want permissions handled at both module level and action level.

Permission layer	What I need	Examples
Module access	Control whether a user can access a module at all.	Customers, orders, billing, email templates, settings, affiliates, support, analytics.

Action access	Control what a user can do inside a module.	View, create, edit, delete, refund, export, resend, impersonate, publish, approve, configure.
Scope access	Control which records or environments a user can touch.	Own workspace only, assigned tickets only, all customers, production settings only for senior admins.
Visibility rules	Hide UI elements and prevent access server-side.	Buttons, tabs, filters, row actions, API endpoints.
Auditability	Log permission changes and high-risk actions.	Role edits, status changes, refunds, impersonation, secret rotations.

User type	Minimum role structure	Main access	Notes
Admin users	Super admin, operations admin, support admin, marketing/content admin, finance admin, developer/admin technical role.	Internal admin modules based on role.	Exact naming can change, but the structure must support multiple internal roles.
Customer users	Workspace owner, admin/manager, standard member, billing contact, affiliate user if used.	Customer app modules based on role.	Customer roles are separate from admin roles.
Invited users	Pending invited team members with limited pre-acceptance state.	Invitation acceptance flow only.	Need invite expiry and resend logic.
Suspended or restricted users	Users with reduced access due to billing, abuse, or manual restrictions.	Limited or blocked access based on rule.	State changes must be visible to admin and logged.

8. Core data entities I expect the build to support

Entity	What it represents	Linked areas
Users	All people in the system, including admin users and customer-side users.	Auth, profile, permissions, security, support, notifications.

Workspaces / accounts	Customer organisation or account container.	Plans, team members, usage, settings, product data.
Products and plans	Subscription plans, one-time products, add-ons, and credit packs.	Pricing, checkout, billing, entitlements.
Subscriptions	Live and historical subscription records.	Billing, renewals, cancellations, invoices, app access.
Orders	Checkout/order records across purchase types.	Customers, payments, accounting, support.
Payments / invoices	Payment attempts, settled payments, invoices, receipts, refunds.	Finance, customer billing history, admin actions.
Credits / usage records	Quota, consumed units, top-ups, entitlement snapshots, usage events.	App access, plan limits, billing.
Invitations	Team invites and admin invites.	Team management, onboarding, auth.
Notifications	In-app notices, emails, banners, messages, campaigns.	Customer communication, admin control.
Tickets / support conversations	Customer support records, feedback, bug reports, contact submissions.	Support, customer records, staff workflows.
Affiliates / referrals	Affiliate accounts, referrals, commissions, payouts.	Affiliate system, finance, attribution.
Content items	Website pages, blog posts, legal pages, changelog entries, reusable content blocks.	CMS, SEO, localization.
Settings records	Site settings, auth settings, payment settings, branding settings, feature flags.	Platform configuration.
Audit logs	Sensitive action history.	Security, support, accountability.
Jobs / webhooks	Background tasks, webhook events, retries, failures.	Operations and integrations.
Product-specific entities	Tables specific to the individual SaaS tool.	App product modules.

9. Key workflows that need to work end to end

Workflow	Required flow	Outcome
Visitor to customer	Visitor lands on public site → reads product/pricing → registers or starts checkout/trial → verifies account if required → enters onboarding → reaches app dashboard.	Clean conversion flow from main site into app.
Customer purchase flow	Customer selects plan or product → checkout → payment success/failure handling → entitlement applied → invoice/receipt created → confirmation sent.	Reliable purchase and entitlement assignment.
Trial to paid conversion	User starts trial → sees trial status and limits → adds payment method if required → converts to paid or expires into defined state.	Clear trial lifecycle.
Subscription change flow	User upgrades/downgrades/cancels/resumes → billing rule applied → entitlements updated → records updated → notification sent.	Predictable plan management.
Team invite flow	Workspace owner/admin invites member → email sent → invite accepted → role assigned → access granted → audit log recorded.	Controlled team growth.
Support flow	User opens ticket/contact request → admin receives and manages it → customer receives updates → ticket is closed with history kept.	Operational support handling.
Admin customer management flow	Admin searches customer → views full record → performs allowed action → action is logged → customer-facing state updates if needed.	Safe internal operations.
Email event flow	System event happens → correct template selected → variables rendered → send log saved → delivery status captured if available.	Consistent messaging.

Webhook / background task flow	External event arrives or internal async task runs → process → success or retry/fail handling → logs visible in admin.	Operational reliability.
Content publishing flow	Staff edits content → preview → publish/schedule → public site updates → SEO fields and locale data preserved.	Manageable public content workflow.

10. Billing and subscription rules I want implemented clearly

Billing area	Requirement
Trial support	The system needs to support free trial logic where enabled, including trial start, trial length, trial expiry, conversion, and what happens after expiry.
Upgrade and downgrade rules	Plan changes must handle proration or scheduled change rules cleanly, and the customer must see the effect clearly.
Cancellation	Cancellation must support immediate cancel or cancel-at-period-end depending on project rules.
Failed payments	There needs to be failed payment tracking, retry handling, grace period logic if used, customer notification, and admin visibility.
Payment methods	Customers need to be able to manage payment methods where supported by the gateway.
Invoices and receipts	Invoices/receipts must be stored and visible to the customer and admin.
One-time products	The billing system must support one-time purchases in addition to subscriptions.
Credits and top-ups	If a project uses credits, the system must support buying top-ups, allocating credits, consuming credits, and keeping a full usage history.
Coupons and discounts	Support coupon codes, limited offers, eligibility rules, expiry dates, usage limits, and campaign-specific discounts.
Refunds and manual adjustments	Admin must be able to handle refunds or billing adjustments with logging and permission control.
Taxes / business info	The system should be ready to capture business

	billing data and tax-related fields where needed.
Entitlements	App access must be controlled by active plan, seat count, credits, or other entitlements, not by UI alone.
Seat logic	Where team plans exist, the platform needs seat handling or member limit logic.
Plan archive rules	Old plans/products must be archivable without breaking historical records.
Gateway abstraction	The platform must not be hard-wired to a single gateway in a way that blocks future expansion.

11. Emails, notifications, and communication rules

Communication type	What I need	Typical examples
Transactional emails	System-triggered emails tied to account, billing, support, or security events.	Verify email, welcome, password reset, invite, purchase confirmation, invoice, failed payment, cancellation, ticket update.
Promotional emails	Broadcast or segment-based marketing emails managed from admin.	Newsletters, promotions, launch campaigns, win-back emails.
In-app notifications	Messages shown inside the app or dashboard.	Announcements, billing warnings, maintenance notices, product updates.
Admin alerts	Operational alerts visible to internal users.	Failed jobs, webhook failures, payment failure spikes, abuse/security alerts.
Preference rules	Where needed, customers must have notification preference controls.	Marketing opt-out, channel preferences, non-optional account notices.
Localization	Email templates and app notifications need to support multiple languages if the project is multilingual.	Template variations per locale.

12. Security, reliability, and operations requirements

Area	Requirement
Environments	Use proper separation for local/development, staging, and production.
Domain setup	Each project will use its own domain and subdomains, with Cloudflare handling domain configuration.
Email sending	Platform emails will use AWS SES API/key or equivalent configured in a secure way.
Secrets handling	Secrets, API keys, and provider credentials must not be exposed in the UI in plain text.
Session security	Support secure session handling, invalidation, and device/session review where relevant.
Password and auth safety	Use secure password storage, reset flows, and verification logic.
MFA readiness	Admin and, where needed, customers should be able to use MFA.
Audit logs	Sensitive actions need logging across admin and critical app flows.
Backups	Database and key storage strategy must be suitable for production use.
Error tracking and monitoring	There must be production-grade logging and monitoring hooks.
Queue/job reliability	Async processing must support retry, fail visibility, and replay where safe.
Webhook reliability	Incoming and outgoing webhooks need signature/security handling where applicable plus log visibility.
Rate limiting / abuse controls	Need basic protection for login, auth, public forms, and API endpoints.
Storage handling	File/media storage needs safe deletion rules and stable references.
Performance	Pages, lists, and dashboards must be designed to handle real usage volumes, not only tiny datasets.
Testing	Core flows need enough testing for confidence, especially auth, billing, and permissions.
Accessibility	The UI should follow sensible accessibility practice

	for forms, states, and navigation.
Data retention	Need sensible retention/cleanup rules for logs, invites, and temp records where appropriate.

13. Build order I want followed

Phase	What needs to be completed	Included scope
Phase 1 — foundation	Set up core project architecture and shared services.	Auth base, user/account model, role model, settings base, environment setup, shared UI foundation.
Phase 2 — website	Build the public website and CMS layer.	Main pages, pricing, forms, CMS, SEO fields, localization hooks, auth entry points.
Phase 3 — admin core	Build the internal operations panel.	Admin auth, dashboard, users/roles, customers, orders, subscriptions, products, settings, audit logs.
Phase 4 — app shell	Build the reusable customer shell.	Dashboard, onboarding, profile, billing, subscription, teams, notifications, support, security.
Phase 5 — product modules	Add the project-specific SaaS functionality inside the app.	Actual product workflows, screens, and data structures.
Phase 6 — operations and finish	Complete operational tooling and handover.	Jobs/webhooks screens, monitoring hooks, QA, documentation, deployment-ready handover.

14. What I expect delivered at the end

I want a complete build, not only screens.

Deliverable	What I expect
Codebase	Full codebase for the website, admin, app, and shared platform services.
Database	Schema, migrations, and clean structure for shared entities and product-specific entities.
Integrations	Configured integration layer for AWS SES, payment

	gateways, auth providers, analytics hooks, and other enabled services.
Admin and app route map	Clear route or page map for website, admin, and app areas.
Environment documentation	List of environment variables, provider keys, and setup requirements.
Deployment notes	Instructions for deploy/update/rollback and environment setup.
Permissions map	Clear definition of admin roles and customer roles.
Billing logic documentation	Clear explanation of plan, subscription, credit, and entitlement behaviour.
Basic maintenance guidance	Enough documentation for future maintenance, not just the initial launch.
Reusable structure	A code structure that makes future SaaS projects faster to launch using the same base.

15. Final build instruction

What I want built

Build this as a reusable SaaS platform boilerplate that I can use across multiple projects. The public website, admin panel, and customer app all need to be part of one coherent platform. Standard SaaS features such as auth, onboarding, billing, teams, permissions, content management, emails, notifications, support, tracking, settings, logs, and operational tooling must be included as part of the base. Product-specific functionality will plug into this foundation on a per-project basis, but the shared platform itself needs to be complete and production-ready.